

NAME(S):

**Check your understanding: complete what you don't get to in class in preparation for upcoming exams and bring any questions you have to class next week to get answers.**

1. What is the value of x after this code runs?

```
x = 5
if x > 2:
    x = -3
    x = 1
else:
    x = 3
    x = 2
```

2. What is the value of x after this code runs?

```
x = 1
if x > 2:
    x = -3
    x = 1
else:
    x = 3
    x = 2
```

3. Given the function definition below, what will be printed to the screen when this code runs?

```
def is_odd(x):
    if x % 2 == 1:
        return True
    else:
        return False

print(is_odd(1))
print(is_odd(2))
print(is_odd(3.5))
print(is_odd("abc"))
```

4. Consider the following function definition for is\_odd. Can you rewrite the function so that it only uses one line after "def is\_odd(x):"? If so, provide an example.

```
def is_odd(x):
    if x % 2 == 1:
        return True
    else:
        return False
```

5. What gets printed when the following code runs?

```
grade = 100
if (grade >= 90):
    print("You got an A!")
if (grade >= 80):
    print("You got a B!")
else:
    print("You got another grade.")
```

6. What gets printed when the following code runs?

```
grade = 100
if (grade >= 90):
    print("You got an A!")
elif (grade >= 80):
    print("You got a B!")
else:
    print("You got another grade.")
```

7. Fill in the blanks in the code below so the following payscale is followed.

|     |     | experience (years) |         |         |
|-----|-----|--------------------|---------|---------|
|     |     | 0                  | 1-2     | 3+      |
| age | <18 | \$15.25            | \$16.00 | \$16.00 |
|     | 18+ | \$15.25            | \$16.25 | \$17.50 |

```
if experience == 0:
    wage = 15.25
elif (_1_):
    if (_2_):
        wage = 17.50
    else:
        wage = 16.25
else:
    wage = 16.0
```

8. What will the value of y be after this code runs?

```
x = 8
y = (x > 4) and ((not(x>8))or(x==5))
```

**Hands-On Coding: Conversions**

Save all code for this problem in a file like `conversions.py`. Don't forget to include comments for your code and to organize your code in a reasonable fashion.

1. Write a function called `ft_to_in` that converts an integer indicating a measurement in feet to inches.
  - (1 ft = 12 in)
  - Before writing any code, answer the following questions and include the answers as comments at the top of the file:
    - 1.2 How many parameters does this function need?
    - 1.2 What does each parameter stand for?
    - 1.3 What datatype should the parameter(s) be?
    - 1.4 Does the function have a return value?
      - \* 1.4.1. If so, what is the datatype of the return value?
    - 1.5 Give 2-3 example function calls and the expected output of those calls.
2. Write a function called `in_to_cm` that converts a measurement in inches to centimeters.
  - (1 in  $\approx$  2.54 cm)
  - Think about your responses to the questions from 1. when writing this function.
3. Write a function called `cm_to_m` that converts a measurement in centimeters to meters.
  - (1 m = 100 cm)
  - Think about your responses to the questions from 1. when writing this function.
4. Now write a function called `main` that:
  - 4.1. Asks the user to input a height in feet and inches.
  - 4.2. Uses the functions you defined in 1-3 to calculate the user's height in meters.
  - 4.3. At the end, the function should print out a message to let the user know what their height is in meters.
  - Before writing any code, answer the following questions and include the answers as comments at the top of the file:
    - 4.4. How many parameters does this function need, what do they stand for, and what datatype(s) should they be?
    - 4.5. Does the function have a return value, and if so, what is its datatype?
    - 4.6. Give an example function call and the expected output from the call.
  - Recommendation: Have the user input the feet and inches portion of their height with two separate input calls.

**Hands on coding: Painting**

A paint store wants to build a tool to help their customers estimate how much paint they need to buy and how much labor is going to cost if they hire the store's painters to do the painting.

In your calculations, you can assume that all rooms that need painting are rectangular. All walls of a room as well as its ceiling should be painted. You can disregard any windows and doors the room may or may not have.

1. The paint store has determined that for every 121 square feet of wall space, one gallon of paint is needed. Write a function `paint_estimator` that, given the room's dimensions (length, width, and height) in feet, returns how many gallons of paint are needed.

What subtask do you have to solve for this problem? Define a separate function for it.

2. The paint store has also determined that, for every 121 square feet of wall space, 8 hours of labor will be needed. The company charges \$60 per hour of labor. Define another function called `labor_estimator` that, given the room's dimensions (see previous), returns the labor cost.

If you identified a good subtask in the first part of the problem, you should be able to re-use the function you defined for it.

3. Now write a function called `main` that asks the user to input the dimensions of the room (length, width, and height) in feet. The function should then calculate how many gallons of paint are needed to paint the room and how much the labor will cost. It should print out a message to provide this information to the user.
4. (Optional) Change `main` to randomly generate the dimensions of the room instead of asking the user.

**Hands-On Coding: Overtime pay**

Many companies pay time-and-a-half for any hours worked above 40 in a given week. Write a function called `calculate_wages` that takes two numbers indicating the hours worked (first parameter) and the hourly rate, then calculates the total wages for the week and returns them.

Add some code to call the function. (Optional) Add input calls to ask the user for their hours worked and hourly wage.

**Hands on coding: Road Repair**

Adjusting Reeborg's speed: Reeborg's speed can be adjusted using Reeborg's `think` function, which takes a single parameter `ms`, which indicates how many milliseconds Reeborg should think before taking its next action. This means that the smaller numbers will make Reeborg go faster, while larger numbers will make it go slower. The default value is 250.

1. Download the Road Repair folder (or its contents) from the course website (3 json files for 3 Reeborg worlds) and import them into Reeborg. Reminder: to import a world into Reeborg using a json file, follow these steps:
  - a. Click "Additional options" at the top right to bring up the "Additional options" window.

- b. Click the “Open world from file” button.
  - c. Select the json file you want to import.
2. Each world should consist of a “roadway” with “potholes” in it.
  3. Write a program so Reeborg walks/drives along the road and fills in the potholes as it goes.
    - a. Use Reeborg’s `build_wall` function, which puts a wall in front of Reeborg.
  4. You should write a single program that will solve all three roadX worlds.